

DBMS Interview Study Guide

Eight topic areas, discovered from current interview-report and authoritative-documentation sources rather than a fixed syllabus — spanning core architecture, data modeling, normalization, SQL query writing, NULL semantics, indexing and query plans, transactions and concurrency, and replication/reliability. Every isolation-level and locking claim is checked against the PostgreSQL and MySQL InnoDB manuals directly.

65 questions • Basic → Intermediate → Advanced

8 topic areas

Runnable SQL, EXPLAIN reasoning, concurrency timelines

Sourced from GATE, GfG, LeetCode Discuss, Glassdoor, PostgreSQL & MySQL docs

Updated Sep 2026

WHO THIS IS FOR

Intern

Junior

Mid

Senior

- **Intern/Junior** — expect direct definitions, JOIN types, and basic SQL writing (Sections 1–5).
- **Mid-level** — expect you to read an EXPLAIN plan and reason about an index (Section 6), and to write non-trivial SQL with window functions (Section 4).
- **Senior/Staff/DBA** — expect trade-off framing across concurrency anomalies, isolation levels, and replication/failover decisions (Sections 7–8).

WHAT INTERVIEWERS EVALUATE

- **SQL fluency** — writing correct, efficient queries live, not just recognizing syntax.
- **Data modeling judgment** — normalizing correctly, then knowing exactly when to break the rule.
- **Transaction reasoning** — naming the specific anomaly a schedule produces, not just "race condition."
- **Performance intuition** — reading an EXPLAIN plan and proposing the right index, not a random one.

01 Core DBMS Concepts & Architecture

The academic bedrock every later section assumes: what a DBMS actually is, why it exists instead of raw files, and the vocabulary (ACID, schema levels) that shows up in nearly every subsequent question.

CORE DEFINITIONS

DBMS (Database Management System)

Software that manages data storage, retrieval, and updates while enforcing consistency, security, and concurrent access, sitting between raw storage and applications.

RDBMS (Relational DBMS)

A DBMS built specifically on the relational model — data stored in tables (relations) with rows and typed columns, related to each other via keys.

1.1 BASIC CONCEPT

What is a DBMS, and how does it differ from just storing data in flat files?

A DBMS is software that acts as an interface between raw stored data and the applications/users that need it, providing a controlled way to define, store, query, update, and secure that data. Storing data directly in flat files (CSVs, plain text) technically "works," but leaves the application responsible for everything a DBMS otherwise guarantees: consistent concurrent access when two processes write at once, crash recovery so a half-written update doesn't corrupt the file, efficient lookup without scanning the entire file linearly, referential integrity between related records, and access control. A DBMS centralizes these guarantees once, in one battle-tested layer, instead of every application reimplementing (and likely getting wrong) its own version of concurrency control and crash recovery.

data independence

concurrency control

crash recovery

1.2 BASIC COMPARISON DBMS vs RDBMS — what's the actual difference?

	DBMS	RDBMS
Data model	Any (hierarchical, network, flat file-based)	Strictly relational — tables of rows/columns
Relationships	Not necessarily enforced structurally	Enforced via primary/foreign keys
Normalization	Not a native concept	Core design discipline (Section 3)
ACID guarantees	Not guaranteed by definition	Standard expectation (transactions, Section 7)
Examples	A flat-file system, early hierarchical DBs	PostgreSQL, MySQL, Oracle, SQL Server

Every RDBMS is a DBMS; not every DBMS is relational — the term DBMS is the broader category, and in casual industry conversation "database" almost always implicitly means RDBMS unless someone says "NoSQL."

relational model

referential integrity

1.3 **BASIC** CONCEPT **Explain the three-schema architecture and data independence.**

The three-schema architecture separates how data is presented, structured, and stored into three independent layers: the **external schema** (what a specific application or user sees — a view, a subset of columns), the **conceptual schema** (the full logical structure of the database — all tables, columns, relationships, constraints, independent of any one application), and the **internal schema** (the physical storage details — file organization, indexes, compression). The point of separating them is **data independence**: *logical* data independence means the conceptual schema can change (adding a column) without breaking external-schema applications that don't use it; *physical* data independence means the internal schema can change (adding an index, changing storage engine) without requiring any change to the conceptual schema or the applications built on it at all.

external schema

conceptual schema

internal schema

logical/physical data independence

1.4 **BASIC** CONCEPT **What are the ACID properties, illustrated with a bank-transfer example?**

Consider transferring \$100 from Account A to Account B — a transaction that must debit A and credit B as one indivisible unit.

- **Atomicity** — the transaction is all-or-nothing. If the credit to B fails after the debit from A succeeded, the whole transaction rolls back; A is restored to its original balance. Money can never vanish mid-transfer.
- **Consistency** — the transaction moves the database from one valid state to another, respecting all constraints (e.g. no account balance goes negative if that's a defined rule). Before and after the transfer, total money in the system is unchanged.
- **Isolation** — a concurrently running transaction (say, a bank statement query) never sees the transfer in a half-completed state — either it sees the balances before the transfer or fully after, never A debited but B not yet credited.
- **Durability** — once the transfer commits, it survives a crash immediately afterward. A power failure one millisecond after commit does not undo the transfer.

Atomicity

Consistency

Isolation

Durability

1.5 INTERMEDIATE CONCEPT

What's the difference between a database schema and a database instance?

The **schema** is the structural design — table names, column names and types, constraints, relationships — defined once (via DDL) and changed only deliberately, through migrations. The **instance** is the actual data held in the database at a specific point in time — the rows currently in those tables, which change constantly as the application runs. The relationship is analogous to a class versus an object in programming: the schema is the blueprint, the instance is one snapshot of data conforming to that blueprint at a given moment.

schema instance

1.6 INTERMEDIATE CONCEPT What are DDL, DML, DCL, and TCL?

CATEGORY	PURPOSE	EXAMPLE COMMANDS
DDL (Data Definition)	Defines/alters structure	CREATE TABLE, ALTER TABLE, DROP TABLE
DML (Data Manipulation)	Reads/writes data	SELECT, INSERT, UPDATE, DELETE
DCL (Data Control)	Manages permissions	GRANT, REVOKE
TCL (Transaction Control)	Manages transaction boundaries	COMMIT, ROLLBACK, SAVEPOINT

A commonly missed detail: DDL statements in most RDBMSs (MySQL notably) trigger an **implicit commit** of any open transaction — meaning you generally cannot roll back a CREATE TABLE the way you can roll back an UPDATE, which surprises engineers who assume everything inside a transaction block is equally reversible.

DDL DML DCL TCL implicit commit

	RELATIONAL (SQL)	NOSQL
Schema	Fixed, enforced upfront	Flexible/schemaless, enforced (if at all) by the application
Consistency model	Strong consistency, ACID transactions by default	Often eventual consistency, tunable per operation
Relationships	Native, via JOINS and foreign keys	Typically denormalized/embedded, or handled in application code
Scaling pattern	Traditionally vertical, or sharded with effort	Designed for horizontal scaling from the start
Best fit	Complex relationships, multi-record transactions (financial systems, order management)	Very high write throughput, flexible/evolving data shapes, massive horizontal scale (event logs, catalogs, session stores)

The honest framing interviewers want, rather than "NoSQL is more scalable": relational databases enforce correctness guarantees NoSQL systems often push onto the application in exchange for easier horizontal scaling — it's a trade-off between where complexity lives, not a strict upgrade in either direction.

schema-on-write vs schema-on-read

eventual consistency

Scenario: you're picking a database for a new e-commerce order system. Relational or NoSQL, and why?

Relational is the defensible default here. An order touches inventory counts, payment state, customer records, and order line items — several related entities that must change together correctly (decrement inventory *and* record the order *and* charge payment, or none of it) — exactly the multi-record atomicity ACID transactions exist to guarantee. Losing that guarantee (e.g. an eventually-consistent NoSQL store where inventory and order state can briefly disagree) risks overselling inventory or charging a customer for an order that was never actually recorded — a correctness bug with direct financial and legal consequences, not just a UX glitch.

NoSQL becomes the better fit for adjacent parts of the same system that don't need this: the product catalog (flexible, evolving attributes per category, read-heavy, rarely needs multi-record transactions), the shopping cart/session store (high write volume, naturally per-user partitioned, tolerates eventual consistency fine), and clickstream/analytics events (huge volume, no relational structure needed). This is the standard **polyglot persistence** answer interviewers are listening for: not "pick one database for everything," but matching each subsystem's actual consistency and scale requirements.

polyglot persistence

multi-record atomicity

REAL INTERVIEW SIGNAL

Search sources on data-engineer interviews consistently note that "SQL vs NoSQL, justify with a real system" is used specifically to separate candidates who've internalized the trade-off from those repeating "NoSQL scales better" as a slogan.

WHAT THE WEB SOURCES EMPHASIZE

GeeksforGeeks and DataCamp's DBMS question sets both structure their material around the same four-part progression — foundational concepts/ACID, SQL syntax, performance/scale, and real-world scenario handling — which directly shaped how this guide's eight sections are sequenced.

02 Data Modeling, ER Diagrams & Keys

Before any SQL is written, a schema has to be designed. This section covers the ER-modeling vocabulary and key types that every design question in later sections assumes.

2.1 BASIC CONCEPT

What is an ER diagram, and what do entities, attributes, and relationships represent?

An Entity-Relationship diagram is a visual modeling tool used before physical schema design to capture what a system needs to store and how those things relate. An **entity** is a distinct real-world object or concept the system tracks (a Customer, a Product, an Order) — it becomes a table. An **attribute** is a property of an entity (a Customer's email, a Product's price) — it becomes a column. A **relationship** describes how entities are associated (a Customer *places* an Order) — depending on its cardinality (covered in 2.4), it becomes either a foreign key on one table or an entirely new junction table. ER modeling is deliberately done before writing SQL because catching a modeling mistake on a whiteboard costs a conversation; catching it after a schema has real production data costs a migration.

entity attribute relationship cardinality

2.2 BASIC COMPARISON

Primary key vs candidate key vs super key vs alternate key vs composite key.

KEY TYPE	DEFINITION
Super key	Any set of columns that uniquely identifies a row (may include extra, unnecessary columns)
Candidate key	A <i>minimal</i> super key — no column can be removed without losing uniqueness
Primary key	The one candidate key chosen to be the table's official identifier
Alternate key	Any candidate key <i>not</i> chosen as primary (still enforced unique, e.g. via a UNIQUE constraint)
Composite key	A candidate/primary key made of more than one column together

Example: on a Users table, both `user_id` and `email` could each independently identify a row — both are candidate keys; if `user_id` is chosen as primary, `email` becomes an alternate key, still enforced unique.

super key candidate key alternate key

What is a foreign key, and what do ON DELETE CASCADE/RESTRICT/SET NULL mean?

A foreign key is a column (or columns) in one table that references the primary key of another table, enforcing that any value stored there must actually exist in the referenced table — the mechanism that makes referential integrity a database-enforced guarantee rather than an application convention.

```
CREATE TABLE orders (  
  order_id    INT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
    ON DELETE RESTRICT  
);
```

The ON DELETE clause defines what happens to child rows (orders) when the referenced parent row (a customer) is deleted: **CASCADE** deletes the child rows too (delete a customer, all their orders vanish — dangerous for audit trails); **RESTRICT** (or the default in many systems) blocks the parent delete entirely while child rows exist; **SET NULL** sets the foreign key column to NULL on the child rows, preserving them as orphaned records (requires the column to be nullable). Choosing the wrong one is a real production incident waiting to happen — CASCADE on a financial record is a common, costly mistake.

foreign key

ON DELETE CASCADE/RESTRICT/SET NULL

2.4 INTERMEDIATE DESIGN

What are 1:1, 1:N, and M:N relationships, and how is M:N implemented relationally?

1:1 — one row in A relates to at most one row in B (e.g. a User and their UserProfile) — typically implemented by putting the foreign key on either table with a UNIQUE constraint. **1:N** — one row in A relates to many rows in B, but each B relates to only one A (a Customer has many Orders) — implemented by placing the foreign key on the "many" side (Orders.customer_id). **M:N** — many rows in A relate to many rows in B (Products and Orders — an order contains many products, a product appears in many orders) — this cannot be represented with a single foreign key on either table, and requires a **junction (or associative) table**:

```
CREATE TABLE order_items (  
  order_id INT,  
  product_id INT,  
  quantity INT NOT NULL,  
  PRIMARY KEY (order_id, product_id),  
  FOREIGN KEY (order_id) REFERENCES orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

junction table

associative entity

2.5 INTERMEDIATE COMPARISON Surrogate key vs natural key — when do you prefer each?

A **natural key** is a real-world attribute that's already unique (an email address, a national ID number, a product's SKU). A **surrogate key** is an artificial identifier with no business meaning, generated purely to identify the row (an auto-incrementing integer, a UUID). Surrogate keys are generally preferred as the primary key in practice because natural keys have a habit of turning out not to be as immutable or unique as assumed — an email can be reused after an account is deleted, a national ID scheme can change format across countries, a SKU can be reassigned — and any of those cause painful cascading updates across every foreign key referencing that value. A natural key is instead kept as a UNIQUE-constrained alternate key: enforced for correctness, but not load-bearing as the join key.

surrogate key

natural key

immutability

2.6 INTERMEDIATE CONCEPT **Weak entity vs strong entity — example with an order line item.**

A **strong entity** has its own primary key and independent existence — an Order can exist and be identified without reference to anything else. A **weak entity** cannot be uniquely identified by its own attributes alone and depends on a strong entity (its "owner") for identification — an OrderItem (line item) typically has no meaningful identity outside the specific order it belongs to; its uniqueness comes from the combination of its owning order plus a local discriminator (e.g. (order_id, line_number)), not from a standalone key of its own. This is why OrderItem is frequently modeled with a composite primary key that includes its parent's foreign key, rather than an independent surrogate key — though many production schemas pragmatically give even weak entities a surrogate key for simpler joins, while still enforcing the underlying dependency via a NOT NULL foreign key.

weak entity

identifying relationship

Design a simplified e-commerce schema: Customers, Orders, OrderItems, Products.

```
CREATE TABLE customers (  
  customer_id SERIAL PRIMARY KEY,  
  email       VARCHAR(255) NOT NULL UNIQUE,  
  created_at  TIMESTAMP NOT NULL DEFAULT now()  
);  
  
CREATE TABLE products (  
  product_id SERIAL PRIMARY KEY,  
  sku         VARCHAR(64) NOT NULL UNIQUE,  
  name        VARCHAR(255) NOT NULL,  
  price_cents INT NOT NULL CHECK (price_cents >= 0)  
);  
  
CREATE TABLE orders (  
  order_id     SERIAL PRIMARY KEY,  
  customer_id INT NOT NULL REFERENCES customers(customer_id),  
  status       VARCHAR(20) NOT NULL DEFAULT 'pending',  
  placed_at    TIMESTAMP NOT NULL DEFAULT now()  
);  
  
CREATE TABLE order_items (  
  order_id     INT NOT NULL REFERENCES orders(order_id),  
  product_id   INT NOT NULL REFERENCES products(product_id),  
  quantity     INT NOT NULL CHECK (quantity > 0),  
  unit_price_cents INT NOT NULL, -- price at time of purchase, not a live join to products  
  PRIMARY KEY (order_id, product_id)  
);
```

The one deliberate, interview-worthy design decision: `unit_price_cents` is stored redundantly on `order_items` rather than always joined live from `products.price_cents` — because a historical order must reflect the price the customer actually paid, which must never silently change if the product's current price changes later. This is a small, well-justified denormalization, foreshadowing Section 3.6.

schema design

price-at-time-of-purchase

Why does modeling M:N with two foreign keys directly on one entity break, and what's the fix?

A common beginner mistake: trying to represent "an order contains multiple products" by adding `product_id_1`, `product_id_2`, `product_id_3` columns directly on the `Orders` table. This breaks immediately on multiple fronts: it hard-caps the number of products per order at however many columns were added, it violates **First Normal Form** by encoding a repeating group as numbered columns instead of separate rows (foreshadowing Section 3.2), and it makes even a simple query like "how many total units of product X have been ordered" require awkward, unmaintainable UNION logic across every numbered column instead of one clean aggregate. The fix, as in 2.4, is a proper junction table (`order_items`) with one row per order-product pair — it scales to any number of items per order and keeps every query a normal JOIN/GROUP BY.

repeating groups

1NF violation

WHAT THE WEB SOURCES EMPHASIZE

Data-modeling interview aggregators consistently pair key-type definitions with a practical schema-design exercise rather than testing definitions in isolation — reinforcing that this section should end in a real, runnable schema, not just a glossary.

03 Normalization & Functional Dependencies

GATE's DBMS syllabus and nearly every interview source converge tightly here: functional dependencies through BCNF, and — just as important — knowing when to deliberately break the rules.

3.1 BASIC CONCEPT What is a functional dependency?

A functional dependency $A \rightarrow B$ ("A determines B") means that for any two rows sharing the same value of A, they must also share the same value of B — knowing A's value is enough to know B's value with certainty. Example: in an unnormalized Orders table, `order_id` $\rightarrow$ `customer_email` if each order belongs to exactly one customer with one email — given the `order_id`, the email is fully determined. Functional dependencies are the formal tool used to detect redundancy: if `customer_id` $\rightarrow$ `customer_email` also holds independently, then storing `customer_email` directly on the Orders table (rather than just `customer_id`) is redundant — the same email gets repeated on every order row for that customer, which is exactly the kind of redundancy normalization exists to remove.

functional dependency

determinant

3.2 BASIC CONCEPT Walk through 1NF, 2NF, and 3NF on an unnormalized table.

Start with a single, unnormalized Orders table mixing order, customer, and product data in one row per line item:

```
Orders(order_id, customer_name, customer_email, product_id, product_name, product_price, quantity)
```

- **1NF** — every column must hold a single, atomic value, and there can be no repeating groups. If a real version of this table instead stored a comma-separated list of `product_ids` in one cell per order, that violates 1NF (exactly the failure mode from 2.8) — the fix is one row per (order, product) pair, as already shown.
- **2NF** — requires 1NF, plus no **partial dependency**: every non-key column must depend on the *entire* composite primary key, not just part of it. With primary key (`order_id`, `product_id`), `customer_name` and `customer_email` depend only on `order_id` (not on `product_id` at all) — a partial dependency. Fix: move customer data to its own Customers table.
- **3NF** — requires 2NF, plus no **transitive dependency**: a non-key column can't depend on another non-key column. `product_name` and `product_price` depend on `product_id`, which is itself a non-key attribute of the order line — a transitive dependency ($\text{order_id} \rightarrow \text{product_id} \rightarrow \text{product_name}$). Fix: move product data to its own Products table, leaving only `quantity` (and a snapshot price, per 2.7) on the line-item table.

The end state is exactly the 2.7 schema: Customers, Products, Orders, OrderItems — each in 3NF.

1NF

2NF

3NF

partial dependency

transitive dependency

3.3 **BASIC** CONCEPT **What are insertion, update, and deletion anomalies?**

Using the same unnormalized Orders table from 3.2 as the running example — all three anomalies are direct consequences of the redundancy normalization removes:

- **Insertion anomaly** — you can't record a new product in the catalog until someone actually orders it, because product data only exists attached to an order row. You also can't add a brand-new customer without them having placed an order yet.
- **Update anomaly** — if a customer changes their email, every single order row for that customer must be updated. Miss one, and the same customer now has two different emails on file simultaneously — an inconsistent database with no error raised.
- **Deletion anomaly** — if a customer's only order is deleted (e.g. cancelled and purged), their customer record disappears too, since it was never stored independently — you've lost the customer's contact information as an unintended side effect of removing an order.

insertion anomaly

update anomaly

deletion anomaly

3.4 **INTERMEDIATE** COMPARISON **BCNF vs 3NF — what's the difference, with an example?**

Boyce-Codd Normal Form is a stricter version of 3NF: for every functional dependency $A \rightarrow B$ in the table, A must be a *super key* (3NF has a narrower exception allowing B to be part of a candidate key even if A isn't a super key). The classic textbook case where a table is in 3NF but not BCNF involves overlapping candidate keys: a table Enrollment(student_id, course_id, instructor) where each course has exactly one instructor ($\text{course_id} \rightarrow \text{instructor}$), but a student can take a course from a specific instructor, and instructors can teach multiple courses. If (student_id, course_id) is the primary key, but $\text{course_id} \rightarrow \text{instructor}$ holds and course_id alone is not a super key, this dependency violates BCNF even though it satisfies 3NF's narrower exception. The fix is the same normalization move as always: split out Courses(course_id, instructor) as its own table. In practice, most production schemas that reach 3NF also happen to satisfy BCNF — the gap between them mostly matters for exam questions and for schemas with unusual overlapping-candidate-key structures.

BCNF

overlapping candidate keys

3.5 INTERMEDIATE CONCEPT What is a transitive dependency, and how does 3NF eliminate it?

A transitive dependency exists when a non-key attribute depends on another non-key attribute, which in turn depends on the primary key — formally, if $A \rightarrow B$ and $B \rightarrow C$ (and B is not a candidate key), then $A \rightarrow C$ is transitive. From 3.2: $\text{order_id} \rightarrow \text{product_id} \rightarrow \text{product_name}$ means product_name is transitively dependent on order_id through the non-key column product_id . 3NF eliminates this by requiring every non-key column to depend *directly* and *only* on the key — which forces product_name out into its own Products table, keyed directly by product_id , removing the indirect chain entirely.

transitive dependency

3.6 INTERMEDIATE CONCEPT

What is denormalization, and when is it legitimate rather than a mistake?

Denormalization is the deliberate, informed reintroduction of redundancy into a normalized schema, trading some write-time consistency risk and storage for read-time performance — typically by pre-joining or duplicating data to avoid an expensive JOIN or aggregation at query time. It's legitimate engineering, not a mistake, when the trade-off is made **consciously**: the redundant data has a clear, enforced update path (so it doesn't silently drift out of sync), the read pattern that benefits is genuinely frequent and performance-critical, and the team accepts the added complexity of keeping the duplicate in sync (via application logic, triggers, or an async job). The `unit_price_cents` snapshot from 2.7 is a legitimate example — it's not even really "redundant" data going stale, since a historical order price is supposed to be frozen. A precomputed `customer_lifetime_value` column updated by a nightly batch job is a different, riskier kind of denormalization: legitimate for a dashboard that tolerates being a day stale, illegitimate if used anywhere that needs the true current value.

denormalization

conscious redundancy

Scenario: your analytics dashboard's 6-table join is too slow. Walk through your reasoning.

1. **Confirm it's actually a modeling problem, not a missing index.** Run `EXPLAIN ANALYZE` (Section 6) first — a slow 6-table join is very often an indexing gap (missing index on a foreign key used in the JOIN condition) rather than a schema problem, and reaching for denormalization before checking this is a common, wasteful overcorrection.
2. **Check whether the query is read-heavy and tolerant of staleness.** A dashboard refreshed once an hour is a very different constraint than a live balance check — this determines whether a precomputed, periodically-refreshed summary table is even an acceptable trade-off.
3. **If it is**, denormalize *at the reporting layer*, not the transactional schema: a separate materialized view or a nightly ETL job into a wide, pre-joined reporting table, explicitly kept apart from the normalized transactional tables (Orders, Customers, Products) still used for the actual order-processing system. This way the write-path system of record stays normalized and correct, while the read-path analytics layer gets its performance win without risking the correctness guarantees Section 2.7's design depended on.
4. **If it isn't** (the dashboard needs to be live-accurate), the better fix is closer to the indexing/query-plan work in Section 6, or restructuring the query itself, not denormalization at all.

materialized view

reporting layer vs system of record

What are 4NF and 5NF, and why do most production schemas stop at 3NF/BCNF?

4NF eliminates **multi-valued dependencies** — a situation where one attribute has multiple independent, unrelated multi-valued facts attached to it (e.g. a table tracking both an employee's skills and their children's names in one relation produces a combinatorial cross-product of rows that has nothing to do with any real relationship between skills and children). **5NF** (Project-Join Normal Form) addresses cases where a table can be losslessly decomposed into three or more smaller tables but not just two — a subtle, rarely-encountered structural redundancy. Most production schemas stop at 3NF or BCNF in practice because the anomalies 4NF and 5NF address are genuinely rare in typical business schemas — they mostly arise in specific, unusual modeling situations — while the cost of chasing full 5NF compliance (more tables, more joins for every query) is a real, everyday cost paid on every single query. This is itself a normalization-vs-performance trade-off, made at the schema-design level rather than the denormalization level from 3.6.

4NF

multi-valued dependency

5NF

WHAT THE WEB SOURCES EMPHASIZE

GATE's own DBMS syllabus and question archives explicitly cap rigorous treatment at BCNF and functional dependencies, treating 4NF/5NF as lighter-weight or excluded — matching how lightly production-facing interview sources treat those higher normal forms compared to 1NF–BCNF.

04 SQL Fundamentals & Query Writing

LeetCode Discuss's own analysis of hard SQL interview questions found roughly: window functions 35%, joins/unions 30%, aggregations 15%, CASE/IF 10%, CTEs 10% — this section is weighted accordingly.

4.1 BASIC GOTCHA

What is SQL's logical execution order, and why can't you use a SELECT alias in WHERE?

Although you *write* a query as `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY`, the database *evaluates* it in a different order: **FROM/JOIN** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY** (with `LIMIT` last). This ordering explains a classic gotcha:

```
SELECT price * quantity AS line_total
FROM order_items
WHERE line_total > 100; -- ERROR: Line_total does not exist here
```

This fails because `WHERE` is evaluated *before* `SELECT` — the alias `line_total` doesn't exist yet at the point `WHERE` is checked. The fix is to repeat the expression in `WHERE`, or move the filter to `HAVING` (evaluated after `SELECT`) if it's grouped, or wrap the query in a subquery/CTE. This single piece of execution-order knowledge resolves most "why doesn't my alias work" confusion.

logical query processing order

alias scoping

4.2 BASIC COMPARISON JOIN types — INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF.

JOIN	RETURNS
INNER JOIN	Only rows with a match in both tables
LEFT JOIN	All left-table rows; unmatched right-side columns are NULL
RIGHT JOIN	All right-table rows; unmatched left-side columns are NULL
FULL OUTER JOIN	All rows from both sides; unmatched columns on either side are NULL
CROSS JOIN	Every row of A paired with every row of B (Cartesian product)
SELF JOIN	A table joined to itself (via aliases), for hierarchical/peer comparisons

```
-- Customers with zero orders (a common LEFT JOIN use case)
SELECT c.customer_id, c.email
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- Employees who share a manager (self join)
SELECT a.name, b.name AS peer
FROM employees a
JOIN employees b ON a.manager_id = b.manager_id AND a.employee_id != b.employee_id;
```

INNER/LEFT/RIGHT/FULL join

self join

4.3 BASIC SQL WHERE vs HAVING.

WHERE filters individual rows *before* grouping happens; HAVING filters *groups*, after aggregation — which is why HAVING can reference aggregate functions like COUNT() or SUM() and WHERE cannot.

```
-- Customers with more than 5 orders
SELECT customer_id, COUNT(*) AS order_count
FROM orders
WHERE status = 'completed'      -- filters rows first
GROUP BY customer_id
HAVING COUNT(*) > 5;            -- filters groups after aggregation
```

row filter vs group filter

4.4 INTERMEDIATE SQL Write a query to find the second-highest salary — three approaches.

```
-- 1. Subquery (excludes the max, then takes the new max)
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- 2. OFFSET (simple, but breaks silently on ties)
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- 3. Window function (most robust, handles ties correctly)
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) ranked
WHERE rnk = 2;
```

Approach 3 is the answer that distinguishes candidates: it's the only one of the three that correctly returns the "second distinct salary value" even when multiple employees are tied for the highest salary — `OFFSET 1` would incorrectly return a tied-for-first value in that case, a gotcha worth naming out loud in an interview even if you write approach 1 or 2 first.

DENSE_RANK

tie-handling

Explain window functions, then write a "top-2 orders per customer" query.

A window function computes a value *across* a set of related rows (a "window," defined by `PARTITION BY`) while still returning one row per input row — unlike `GROUP BY`, which collapses rows into one row per group. `ROW_NUMBER()` assigns strictly increasing unique numbers even on ties; `RANK()` gives ties the same rank but skips subsequent numbers (1,1,3); `DENSE_RANK()` gives ties the same rank without skipping (1,1,2); `LAG()`/`LEAD()` read a value from a preceding/following row within the window, without a self-join.

```
SELECT customer_id, order_id, amount
FROM (
  SELECT customer_id, order_id, amount,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY amount DESC
         ) AS rn
  FROM orders
) ranked
WHERE rn <= 2;
```

PARTITION BY

ROW_NUMBER

RANK

LAG/LEAD

4.6 INTERMEDIATE COMPARISON CTE vs subquery vs VIEW.

A **CTE** (WITH clause) names a temporary result set for the duration of one query — mainly a readability tool that lets a complex query be broken into named, sequential steps, and uniquely, supports **recursion**. A **subquery** does the same computation inline, unnamed, which becomes hard to read once nested more than one or two levels deep. A **VIEW** is a named, saved query stored in the database schema itself, reusable across many different queries and sessions — unlike a CTE, which exists only within the single statement that defines it.

```
-- Recursive CTE: an employee's full management chain
WITH RECURSIVE mgmt_chain AS (
  SELECT employee_id, manager_id, 1 AS depth
  FROM employees WHERE employee_id = 42
  UNION ALL
  SELECT e.employee_id, e.manager_id, mc.depth + 1
  FROM employees e
  JOIN mgmt_chain mc ON e.employee_id = mc.manager_id
)
SELECT * FROM mgmt_chain;
```

CTE

recursive CTE

VIEW

4.7

ADVANCED

SQL

Find and delete duplicate rows, keeping one copy.

```
-- Find duplicates (same email, different id)
SELECT email, COUNT(*)
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;

-- Delete all but the lowest-id copy of each duplicate
DELETE FROM customers
WHERE customer_id NOT IN (
    SELECT MIN(customer_id)
    FROM customers
    GROUP BY email
);
```

The window-function alternative avoids the self-referencing-subquery-on-the-same-table restriction some databases impose on DELETE:

```
DELETE FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM (
        SELECT customer_id,
            ROW_NUMBER() OVER (PARTITION BY email ORDER BY customer_id) AS rn
        FROM customers
    ) t WHERE rn > 1
);
```

de-duplication

MIN() trick

4.8 ADVANCED SQL Write month-over-month revenue growth % using window functions.

```
WITH monthly AS (  
  SELECT date_trunc('month', order_date) AS month,  
         SUM(amount) AS revenue  
  FROM orders  
 GROUP BY 1  
)  
SELECT month,  
       revenue,  
       LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,  
       ROUND(  
         100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))  
         / NULLIF(LAG(revenue) OVER (ORDER BY month), 0)  
       , 2) AS growth_pct  
FROM monthly  
ORDER BY month;
```

LAG() pulls the previous row's revenue into the current row without a self-join; NULLIF(..., 0) guards against a division-by-zero error if a prior month had exactly zero revenue — a small but real production detail worth stating explicitly, since a raw divide-by-zero would error the whole query rather than returning NULL for that one row.

LAG for period-over-period

NULLIF divide-by-zero guard

4.9 ADVANCED GOTCHA UNION vs UNION ALL — what silently changes?

UNION combines two result sets and removes duplicate rows, which requires the database to perform an implicit sort/hash-based deduplication pass over the entire combined result — real work that scales with result size. UNION ALL simply concatenates both result sets with no deduplication step at all — cheaper, and the default choice whenever you already know the two sources can't produce overlapping rows (e.g. unioning this year's and last year's partitioned tables) or when duplicates are actually acceptable or even expected. The gotcha: reaching for UNION out of habit on a large query silently costs a full deduplication pass for a guarantee the query may not even need — a frequently-cited, easy performance win once you know to look for it in a slow-query review.

UNION

UNION ALL

implicit deduplication cost

WHAT THE WEB SOURCES EMPHASIZE

The LeetCode Discuss analysis found in search explicitly quantifies difficulty distribution — window functions (35%), joins/unions (30%), aggregations (15%), CASE expressions (10%), CTEs (10%) — which is the direct basis for this section's emphasis and question count per topic.

05 NULL Semantics & SQL Gotchas

The section interviewers use as a filter: correct SQL syntax knowledge is common, but correct NULL reasoning under three-valued logic reliably separates candidates.

5.1 BASIC GOTCHA What is NULL, and why does SQL use three-valued logic?

NULL represents the *absence* of a known value — not zero, not an empty string, not "unknown but comparable" — genuinely unknown or inapplicable. Because of this, SQL comparisons don't just evaluate to TRUE or FALSE; they can evaluate to a third state, **UNKNOWN**, whenever NULL is involved: `NULL = NULL` is UNKNOWN (not TRUE — two unknowns aren't provably equal), `NULL = 5` is UNKNOWN, and even `NULL != 5` is UNKNOWN, not TRUE. A WHERE clause only keeps rows where the condition is TRUE — UNKNOWN rows are silently excluded, exactly like FALSE rows, which is the root cause of nearly every NULL-related surprise in this section. The only operators designed to test for NULL directly, bypassing three-valued comparison logic, are `IS NULL` and `IS NOT NULL`.

three-valued logic

UNKNOWN

IS NULL

5.2 BASIC GOTCHA COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column).

```
SELECT
  COUNT(*)           AS total_rows,           -- every row, NULLs included
  COUNT(discount_code) AS rows_with_discount, -- NULL discount_code values excluded
  COUNT(DISTINCT discount_code) AS unique_discounts -- NULLs excluded, dupes collapsed
FROM orders;
```

`COUNT(*)` counts every row unconditionally. `COUNT(column)` counts only rows where that specific column is non-NULL. `COUNT(DISTINCT column)` additionally collapses duplicate non-NULL values. Confusing the first two is a very common bug: "how many orders had a discount applied" is `COUNT(discount_code)`, not `COUNT(*)` — the latter silently includes every order, discounted or not.

COUNT(*)

COUNT(column) excludes NULL

Why does NOT IN silently return zero rows when the subquery contains a NULL?

```
-- Intent: customers who have never ordered product 5
SELECT customer_id FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders WHERE product_id = 5
);
-- If ANY row in that subquery has customer_id = NULL, this returns ZERO rows — always.
```

NOT IN (a, b, NULL) expands, under the hood, to $x \neq a$ AND $x \neq b$ AND $x \neq \text{NULL}$. The last comparison, per 5.1, evaluates to UNKNOWN for *every single row* — and TRUE AND UNKNOWN is UNKNOWN, not TRUE — so the entire WHERE condition becomes UNKNOWN for every row, and the whole query silently returns nothing, with no error raised. The fix is NOT EXISTS, which uses existence semantics rather than value comparison and is unaffected by NULLs in the subquery:

```
SELECT c.customer_id FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id AND o.product_id = 5
);
```

NOT IN + NULL trap

NOT EXISTS

REAL INTERVIEW SIGNAL

Multiple search sources independently flag the NOT IN/NULL trap as the single most common SQL "gotcha" question — candidates who've been burned by it in production consistently give the sharpest answers.

5.4 INTERMEDIATE GOTCHA

Why do NULLs never match in a JOIN, and how does that power the LEFT JOIN ... IS NULL anti-join pattern?

A join condition like `ON a.customer_id = b.customer_id` is just another comparison subject to 5.1's rules — if either side is NULL, the comparison is UNKNOWN, and the row pair is never matched, ever, regardless of whether both sides happen to be NULL. This is precisely what makes the anti-join pattern from 4.2 reliable: a `LEFT JOIN` always keeps every left-side row, filling unmatched right-side columns with NULL; filtering `WHERE right_table.id IS NULL` afterward isolates exactly the left rows that found *no* match at all — because a genuine match would have filled that column with a real, non-NULL value. If NULLs *did* match each other in joins, this pattern would break, since a left row with a NULL join-key could then "falsely" match a right row that also happened to have a NULL join key.

NULLs never join-match

anti-join pattern

5.5 INTERMEDIATE GOTCHA How do GROUP BY and DISTINCT treat NULL?

Unlike equality comparisons, `GROUP BY` and `DISTINCT` use a different notion of "sameness" for grouping purposes — they treat all NULLs as belonging to *one single group*, even though `NULL = NULL` is UNKNOWN under ordinary comparison rules (5.1). So `SELECT DISTINCT discount_code FROM orders` returns one row for NULL alongside the other distinct real values, and `GROUP BY discount_code` puts every NULL-discount order into one group together — a deliberate, standard-mandated special case, not a bug, but one that regularly surprises engineers who've just internalized "NULL never equals NULL" from 5.1 and assume it extends everywhere.

GROUP BY groups NULLs together

How would you correctly compute "% of orders with no discount," given NULLs?

```
-- WRONG: COUNT(*) in the denominator looks right but the numerator is the bug-prone part
SELECT
  100.0 * SUM(CASE WHEN discount_code IS NULL THEN 1 ELSE 0 END) / COUNT(*)
  AS pct_no_discount
FROM orders;
```

The subtle trap this question is testing: a naive first instinct is $100.0 * (\text{COUNT}(\text{discount_code}) - \text{COUNT}(\text{discount_code} \text{ IS NULL})) / \text{COUNT}(\text{discount_code})$, relying on the fact that `COUNT(column)` excludes NULLs (5.2) to infer the NULL count by subtraction — which is actually correct, but fragile and easy to get backwards under interview pressure (subtracting the wrong count, or forgetting `COUNT(*)` and `COUNT(discount_code)` behave differently at all). The `CASE WHEN ... IS NULL` version above is preferred specifically because it makes the NULL-counting logic explicit and readable rather than relying on an implicit property of `COUNT()` that's easy to misremember — the kind of defensive, explicit SQL an interviewer wants to see once NULLs are in play.

explicit NULL counting

CASE WHEN aggregation

A report's customer order count dropped after adding a LEFT JOIN. Debug it.

```
-- Suspect query
SELECT c.customer_id, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'completed' -- the bug is right here
GROUP BY c.customer_id;
```

The intent was almost certainly "keep all customers, even ones with zero completed orders" — but that intent has been silently defeated. A `LEFT JOIN` preserves unmatched left rows by filling the right side with `NULL` — so for a customer with no orders at all, `o.status` is `NULL`. The `WHERE o.status = 'completed'` filter then runs, and per 5.1, `NULL = 'completed'` is `UNKNOWN`, so that row is dropped by `WHERE` exactly as if it had failed a normal `FALSE` condition — silently converting the `LEFT JOIN` back into behavior equivalent to an `INNER JOIN`. The fix is to move the status filter into the `JOIN` condition itself, so it's applied *before* the `LEFT JOIN` decides which rows to keep, rather than after:

```
LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status = 'completed'
```

This single bug — filtering a `LEFT JOIN`'s right-table column in `WHERE` instead of in the join condition — is one of the most common real-world sources of a silently wrong report, precisely because the query still runs without error and returns plausible-looking (just wrong) numbers.

WHERE silently defeats LEFT JOIN

filter in ON vs WHERE

WHAT THE WEB SOURCES EMPHASIZE

Multiple independent sources (AlgoMaster, SQL Quest, CliffsNotes) converge on the exact same short list of `NULL` traps — the `NOT IN` trap, `JOIN` non-matching, and `GROUP BY`'s special-case `NULL` grouping — strongly suggesting these three are the actual, recurring interview set rather than an exhaustive list of everything `NULL`-related.

06 Indexing & Query Performance

Search sources are explicit: above junior level, indexing questions test whether you understand *why* a lookup is fast, not whether you can type `CREATE INDEX`.

6.1

BASIC

CONCEPT

What is an index, and why does it speed up lookups?

Without an index, finding rows matching a condition requires a **sequential (full-table) scan** — reading every row, one by one, checking each against the condition — $O(n)$ work regardless of how selective the condition is. An index is a separate, sorted (typically B-Tree, Section 6.2) data structure that maps column values to the physical location of the rows containing them, letting the database narrow the search space logarithmically instead of linearly — the same reason a sorted phone book lets you find a name in a handful of comparisons rather than reading every entry. The trade-off: an index must itself be kept up to date on every INSERT/UPDATE/DELETE that touches an indexed column, so indexes speed up reads at a real cost to write throughput and storage — never a free performance upgrade.

sequential scan

B-Tree

read/write trade-off

6.2

BASIC

COMPARISON

B-Tree vs Hash index.

	B-TREE	HASH
Supports	Equality <i>and</i> range queries (<, >, BETWEEN), ORDER BY, prefix matching	Equality only (=)
Data order	Kept sorted	No inherent order
Typical use	Default index type in virtually every RDBMS	Specific equality-only lookup cases (e.g. some key-value patterns)

B-Tree is the default in PostgreSQL, MySQL/InnoDB, and most RDBMSs precisely because real query workloads mix equality and range predicates constantly — a hash index's $O(1)$ equality lookup is faster in the narrow case it supports, but can't serve a range query or an ORDER BY at all, which is why it's the exception, not the default.

B-Tree

hash index

range query support

6.3 BASIC CONCEPT **Clustered vs non-clustered index.**

A **clustered index** determines the actual physical storage order of the table's rows — there can be only one per table, since rows can only be physically sorted one way (in InnoDB, the primary key is always the clustered index). A **non-clustered (secondary) index** is a separate structure holding indexed column values alongside a pointer/reference back to the actual row — a table can have many. Looking up a row via a non-clustered index in InnoDB typically requires a second step (a "bookmark lookup" back to the clustered index/primary key to fetch the remaining columns) unless the index itself already contains everything the query needs — which is exactly what a covering index (6.4) avoids.

clustered index

secondary index

bookmark lookup

6.4 INTERMEDIATE CONCEPT **What is a covering index?**

A covering index includes every column a specific query needs — both the columns in WHERE/JOIN and the columns in SELECT — so the database can satisfy the entire query directly from the index structure itself, never touching the underlying table at all (an "index-only scan"). This avoids exactly the bookmark-lookup cost described in 6.3.

```
-- Query
SELECT customer_id, order_date FROM orders WHERE status = 'completed';

-- A covering index for exactly this query
CREATE INDEX idx_orders_status_covering
  ON orders (status) INCLUDE (customer_id, order_date);
```

The trade-off: a covering index is wider (more columns) than a minimal index would be, costing more storage and more write overhead to maintain — it's a deliberate optimization for a specific, known, frequently-run query pattern, not a default choice for every index.

covering index

index-only scan

6.5 INTERMEDIATE SQL What is a composite index, and why does column order matter?

A composite (multi-column) index is built on more than one column together, sorted first by the first column, then by the second within each value of the first, and so on. The **leftmost-prefix rule**: the index can be used efficiently for queries filtering on the first column alone, or the first + second together, but generally *cannot* be used efficiently to filter on the second column alone, skipping the first.

```
CREATE INDEX idx_orders_customer_date ON orders (customer_id, order_date);

-- Uses the index efficiently (leftmost prefix: customer_id)
SELECT * FROM orders WHERE customer_id = 42;
-- Uses the index efficiently (full prefix: both columns)
SELECT * FROM orders WHERE customer_id = 42 AND order_date > '2026-01-01';
-- Cannot use this index efficiently - order_date isn't the leftmost column
SELECT * FROM orders WHERE order_date > '2026-01-01';
```

The practical implication: column order in a composite index should be chosen based on actual query patterns — put the column most often filtered alone, or with the highest selectivity, first.

composite index

leftmost-prefix rule

6.6 ADVANCED DEBUG A query on an indexed column is still doing a sequential scan. Why?

- **Low selectivity** — if the indexed column has few distinct values relative to table size (e.g. a status column that's 90% 'completed'), the query planner may correctly conclude a sequential scan is actually cheaper than the overhead of thousands of individual index lookups plus bookmark lookups.
- **Function or expression on the indexed column** — WHERE UPPER(last_name) = 'SMITH' generally cannot use a plain index on last_name, since the stored, sorted values are the raw names, not their uppercased form; a functional index on UPPER(last_name) is required.
- **Implicit type mismatch** — comparing an indexed integer column against a string literal can force an implicit cast on every row, defeating the index.
- **Stale statistics** — the query planner's row-count/selectivity estimates come from periodically collected statistics (e.g. ANALYZE in Postgres); badly stale statistics after a large bulk load can lead the planner to misjudge and pick a scan.
- **Small table** — for a table small enough to fit in a few disk pages, the planner may correctly decide a full scan is cheaper than the overhead of an index lookup at all.

selectivity

functional index

stale statistics

```
EXPLAIN ANALYZE
```

```
SELECT * FROM orders WHERE customer_id = 42;
```

```
-- Sample output:
```

```
Index Scan using idx_orders_customer_id on orders
```

```
(cost=0.42..8.44 rows=3 width=64)
```

```
(actual time=0.021..0.024 rows=3 loops=1)
```

```
Index Cond: (customer_id = 42)
```

```
Planning Time: 0.089 ms
```

```
Execution Time: 0.041 ms
```

- **Index Scan** vs **Seq Scan** vs **Index Only Scan** — the access method actually chosen; Seq Scan on a large, selective query is the first thing to investigate.
- **cost=0.42..8.44** — the planner's own *estimated* cost (startup..total, in arbitrary planner units, not milliseconds) — used to compare candidate plans against each other, not an absolute time.
- **rows=3** (in the cost line) — the planner's *estimated* row count, versus **actual ... rows=3** — the real row count once the query actually ran. A large gap between estimated and actual rows is the single strongest signal of stale statistics (6.6) misleading the planner.
- **actual time=0.021..0.024** — real wall-clock time (startup..total, in milliseconds) for this specific step, from ANALYZE actually executing the query rather than just estimating.

The interview-grade reading skill isn't memorizing this output — it's knowing to compare estimated vs actual rows first, and to look for Seq Scan on a plan where an index should apply.

EXPLAIN ANALYZE

estimated vs actual rows

6.8 ADVANCED DEBUG What's the trade-off of adding many indexes to a heavily-written table?

Every index must be updated synchronously on every INSERT/UPDATE/DELETE that touches an indexed column — so a table with 15 indexes pays 15x the index-maintenance cost on every write, directly slowing down write throughput and increasing lock contention (Section 7) during high-write periods. It also costs real, often underestimated storage — a heavily-indexed table can have its indexes collectively larger than the table's actual data. The decision framework: keep indexes that support genuinely frequent, performance-critical query patterns (confirmed via actual query logs or `EXPLAIN``, not guesswork), and periodically audit for indexes with near-zero actual usage (most RDBMSs expose per-index usage statistics, e.g. Postgres's `pg_stat_user_indexes`) — an unused index is pure write-cost with no offsetting read benefit, and should be dropped.

write amplification

index usage statistics

WHAT THE WEB SOURCES EMPHASIZE

HelloInterview's system-design material and multiple indexing-focused blog posts converge on the same framing: interviewers above junior level use indexing questions to probe understanding of the underlying B-Tree mechanism and selectivity reasoning, explicitly not just whether a candidate remembers `CREATE INDEX` syntax.

07 Transactions & Concurrency Control

Checked directly against the PostgreSQL and MySQL InnoDB manuals. This is the section where "I know what a race condition is" gets replaced by naming the specific anomaly and the specific fix.

7.1 BASIC CONCEPT

Define ACID again, this time tied to the mechanism each property depends on.

Revisiting 1.4 with the underlying machinery named: **Atomicity** is implemented via the transaction log/undo mechanism — on rollback or crash, incomplete changes are unwound using logged before-images. **Consistency** is enforced by constraint checking (foreign keys, CHECK constraints, uniqueness) at commit time. **Isolation** is implemented via concurrency control — either locking or, as in PostgreSQL, **MVCC** (7.6). **Durability** is implemented via the **Write-Ahead Log** (Section 8.1) — changes are flushed to durable log storage before a commit is acknowledged to the client, so a crash immediately after commit can still replay the WAL to recover the change.

WAL for durability

MVCC/locking for isolation

7.2 BASIC CONCEPT **What is a schedule, and what makes it serializable?**

A schedule is the actual, interleaved order in which operations from multiple concurrent transactions execute. A schedule is **serializable** if its outcome is equivalent to *some* serial (one-at-a-time, no interleaving) execution of the same transactions — the gold-standard correctness bar for concurrent execution: the database is free to interleave operations for performance, as long as the observable result is indistinguishable from having run them one after another in some order. Isolation levels below Serializable (7.5) are deliberate, named relaxations of this guarantee, trading strict correctness for concurrency/performance, and each relaxation permits specific, well-defined anomalies.

schedule

serializability

7.3

INTERMEDIATE

CONCURRENCY

Walk through a dirty read with a transaction timeline.

Time	T1 (uncommitted)	T2
t1	BEGIN;	
t2	UPDATE accounts SET balance = 500 WHERE id = 1; -- was 1000	
t3		BEGIN;
t4		SELECT balance FROM accounts WHERE id = 1; -- reads 500!
t5	ROLLBACK; -- balance reverts to 1000	
t6		-- T2 already acted on 500, a value -- that never really existed

A dirty read occurs when a transaction reads a value written by another transaction that has not yet committed — and that write is later rolled back, meaning the reader saw a value that, from the database's true committed history, never actually existed. This is the most severe common anomaly and is only permitted under **Read Uncommitted**, the weakest isolation level.

dirty read

Read Uncommitted

Walk through a lost update, and show the **SELECT FOR UPDATE** fix.

Time	T1	T2
t1	BEGIN;	
t2	SELECT stock FROM products WHERE id = 7; -- reads 10	
t3		BEGIN;
t4		SELECT stock FROM products WHERE id = 7; -- also reads 10
t5	UPDATE products SET stock = 9 WHERE id = 7; -- 10 - 1	
t6	COMMIT;	
t7		UPDATE products SET stock = 9 WHERE id = 7; -- ALSO 10 - 1, overwrites T1's
t8		COMMIT;
	-- Two units were sold, but stock only dropped by 1. T1's decrement is "lost."	

A lost update happens when two transactions read the same value, then both write back an update based on that stale read — the second write silently overwrites the first, and one of the two real changes is lost entirely, with no error raised. The fix, **SELECT ... FOR UPDATE**, acquires a row-level exclusive lock at read time, forcing T2's read to *block* until T1 commits — so T2 reads the true post-update value (9, not 10) and correctly computes 8:

```
BEGIN;
SELECT stock FROM products WHERE id = 7 FOR UPDATE; -- blocks concurrent writers
UPDATE products SET stock = stock - 1 WHERE id = 7;
COMMIT;
```

lost update

SELECT FOR UPDATE

7.5 INTERMEDIATE COMPARISON Isolation levels — which anomalies each permits.

ISOLATION LEVEL	DIRTY READ	NON-REPEATABLE READ	PHANTOM READ
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible (SQL standard) / Prevented (PostgreSQL's actual MVCC implementation)
Serializable	Prevented	Prevented	Prevented

The PostgreSQL-specific footnote in that table is a genuine, documented, interview-worthy detail: the SQL standard permits phantom reads at Repeatable Read, but PostgreSQL's actual MVCC snapshot implementation (7.6) prevents them anyway at that level — stronger than the standard strictly requires, which is a good example of "the spec is the floor, not necessarily the ceiling" for a specific vendor.

Read Uncommitted/Committed

Repeatable Read

Serializable

7.6 ADVANCED CONCURRENCY

What is MVCC, and how does it let PostgreSQL avoid readers blocking writers?

Multi-Version Concurrency Control, per PostgreSQL's own documentation, works by never overwriting a row in place on UPDATE — instead, a new version of the row is created, and old versions are retained (until vacuumed) so that a transaction can be given a consistent snapshot of the database "as of" a specific point in time, regardless of what other transactions are concurrently writing. Because a reader is simply handed an appropriate existing snapshot version rather than needing to wait for a writer's lock to release, and a writer creates a new row version rather than needing to wait for readers to finish with the old one, **locks acquired for reading never conflict with locks acquired for writing** — reads never block writes, and writes never block reads. This is a structural alternative to purely lock-based concurrency control (like classic two-phase locking), not just an optimization on top of it — it's the reason PostgreSQL handles read-heavy-plus-write-heavy mixed workloads well without readers and writers constantly queuing behind each other.

MVCC

snapshot

readers never block writers

Explain write skew — why doesn't Repeatable Read prevent it, but Serializable does?

```
-- Invariant: at least 1 doctor must be on call at all times
Time  T1 (Dr. A goes off call)          T2 (Dr. B goes off call)
-----
t1    BEGIN;                          BEGIN;
t2    SELECT COUNT(*) FROM doctors
      WHERE on_call = true; -- reads 2 (A and B)
t3                                SELECT COUNT(*) FROM doctors
                                WHERE on_call = true; -- also reads 2
t4    -- 2 >= 1, safe to go off call
      UPDATE doctors SET on_call = false
      WHERE name = 'A';
t5                                -- 2 >= 1, safe to go off call
                                UPDATE doctors SET on_call = false
                                WHERE name = 'B';
t6    COMMIT;
t7                                COMMIT;
-- Both commit successfully. Zero doctors are now on call. Invariant broken.
```

Write skew occurs when two transactions each read an overlapping set of data, each independently verify some invariant holds, and then each write to *different* rows based on that read — neither transaction directly conflicts with the other's write (they touch different rows: A's row vs B's row), so standard row-level locking and even Repeatable Read's snapshot isolation don't detect any conflict at all — both transactions see a valid pre-condition and both commit, yet the combined effect violates an invariant neither transaction could see in isolation. True **Serializable** isolation (in PostgreSQL, Serializable Snapshot Isolation, per its own docs) detects this by tracking read/write dependencies between concurrent transactions more broadly than just row-level write conflicts, and will abort one of the two transactions with a serialization failure, forcing a retry — the only isolation level that actually closes this gap.

write skew Serializable Snapshot Isolation

Per the MySQL manual: a **next-key lock** is a combination of a record lock on an index row plus a **gap lock** on the space immediately before it. By default, InnoDB runs at Repeatable Read, and uses next-key locks specifically to prevent **phantom reads** on range scans — without locking the "gap" between existing index entries, a concurrent transaction could insert a new row into that gap that would then appear if the first transaction re-ran its range query, violating repeatable-read semantics for range predicates, not just single rows.

This is also a common, documented source of InnoDB deadlocks: two transactions inserting into the same index gap in a different order can each hold a gap lock the other needs. The documented debugging path: run `SHOW ENGINE INNODB STATUS` for the last deadlock's detail, or query `performance_schema.data_locks / data_lock_waits` for live visibility into exactly which locks are held and awaited. Documented mitigation strategies: acquire locks in a consistent order across all transactions, keep transactions short, and — if phantom protection isn't actually needed for a given workload — switching to Read Committed disables gap locking entirely, trading that protection for significantly reduced lock contention on range queries.

[gap lock](#)[next-key lock](#)[SHOW ENGINE INNODB STATUS](#)

Two checkout requests concurrently decrement inventory. Compare pessimistic and optimistic fixes.

This is the 7.4 lost-update pattern in a concrete production shape: two customers checking out the last unit of stock simultaneously, both reading `stock = 1`, both deciding it's available, both decrementing — resulting in stock at -1 and two customers charged for one unit of inventory.

```
-- Pessimistic: Lock the row up front, second transaction waits
BEGIN;
SELECT stock FROM products WHERE id = 7 FOR UPDATE;
-- application checks stock > 0 in code, then:
UPDATE products SET stock = stock - 1 WHERE id = 7;
COMMIT;

-- Optimistic: no lock held; the WHERE clause itself is the safety check
UPDATE products
SET stock = stock - 1
WHERE id = 7 AND stock > 0;
-- application checks: if rows_affected = 0, sold out — no stock was actually available
```

Pessimistic locking (`FOR UPDATE`) is simpler to reason about and guarantees correctness by serializing access up front, at the cost of one transaction blocking behind the other — fine for low-contention resources, a bottleneck for a genuinely hot single row (a flash-sale item every customer is hitting simultaneously). **Optimistic concurrency** (the conditional `WHERE stock > 0` version, or more generally a version-number/compare-and-swap column checked on update) avoids holding any lock at all — the database's own atomic row-update guarantee does the safety check, and the application just checks whether the update actually affected a row — trading a small amount of application-side complexity (handling the "0 rows affected" case) for much better throughput under high contention.

pessimistic locking

optimistic concurrency

compare-and-swap update

WHAT THE WEB SOURCES EMPHASIZE

PostgreSQL's own documentation was the clearest authority found on MVCC and isolation-level behavior (including the phantom-read footnote at Repeatable Read), while MySQL's reference manual was equally explicit and current on gap/next-key locking — both used directly rather than paraphrased from secondary sources, per the assignment's authoritative-docs requirement.

08 Reliability, Replication & Scaling

Where a database stops being a query engine and starts being a system that must survive a crash, a network partition, or a 2am page.

8.1 BASIC CONCEPT

What is a Write-Ahead Log, and why write to it before the actual data files?

Per PostgreSQL's own documentation, the Write-Ahead Log records every change to the database's data files sequentially, and — critically — every change is written to the WAL *before* it's applied to the actual data pages. This ordering is what makes crash recovery possible: if the system crashes between writing to the WAL and applying the change to the data files, recovery simply replays the WAL entries after the last checkpoint to bring the data files up to date — the log is the durable source of truth, and the data files can always be reconstructed from it. Writing sequentially to a log is also much faster than writing scattered random updates directly to data pages, which is a secondary but real performance benefit of the design.

Write-Ahead Log (WAL)

checkpoint

crash recovery

8.2 BASIC RELIABILITY Full backup vs incremental backup vs point-in-time recovery.

A **full backup** captures the entire database at one point in time — simple to restore from, but expensive to take frequently at scale. An **incremental backup** captures only what changed since the last backup — cheaper to take often, but restoring requires replaying the full backup plus every incremental since. **Point-in-Time Recovery (PITR)**, per PostgreSQL's documentation, combines a file-system-level base backup with the continuously archived WAL: to recover, you restore the base backup, then replay archived WAL records — and crucially, you can stop replay at *any* point, not just the moment of the last backup, letting you restore to the exact second before an accidental `DROP TABLE` rather than losing everything back to the last nightly backup.

full backup

incremental backup

PITR

Synchronous vs asynchronous replication — the consistency/availability trade-off.

In **synchronous replication**, the primary waits for at least one replica to confirm it has received (and possibly applied) a write before acknowledging that write as committed to the client — this guarantees zero data loss on failover (the replica is always caught up), at the cost of added write latency and reduced availability (if the replica is unreachable, writes can stall or fail entirely). In **asynchronous replication**, the primary acknowledges the write immediately and ships it to replicas in the background — lower latency and better availability, at the real cost that a primary failure can lose whatever writes hadn't yet been shipped and applied to the replica at the moment of failure. This is a direct, concrete instance of the CAP-theorem-style trade-off (8.6) made explicit at the replication-configuration level, not just an abstract theorem.

synchronous replication

asynchronous replication

What does "losing a tiny fraction of data" mean concretely in an async replication failover?

Concretely: as each WAL record is written and acknowledged on the primary, it's streamed to the replica and applied there — but there's always some window of WAL records that have been committed on the primary but not yet shipped, or shipped but not yet applied, on the replica (this window is called **replication lag**, and can be seconds under normal load, or much larger if the replica has fallen behind). If the primary fails and the system fails over to that replica, every write inside that unshipped window — the specific set of committed transactions the replica never received — is permanently lost, even though the client that made those writes was already told they succeeded. This is the exact, mechanical reason "asynchronous replication risks data loss on failover" isn't an abstract warning, but a specific, bounded, measurable window tied directly to current replication lag.

replication lag

unshipped WAL window

8.5 INTERMEDIATE DESIGN What is sharding, and what makes a good vs bad shard key?

Sharding splits one logical table's rows across multiple physical database instances, each holding only a subset of the data, based on a **shard key** — enabling horizontal scaling past what a single machine can hold or serve. A good shard key distributes both storage and query load evenly and lets most real queries be satisfied by a single shard. For the e-commerce schema from Section 2: sharding Orders by `customer_id` is generally good — most real queries ("this customer's order history") naturally target one customer and therefore one shard, and customers are numerous enough to distribute load evenly. Sharding by `order_date` is a common trap — it looks reasonable, but concentrates *all* of today's write traffic onto whichever single shard holds "today," creating a hot shard, while historical shards sit nearly idle — the opposite of the even distribution sharding is supposed to provide. The general lesson: a shard key should be chosen from the dimension real traffic is naturally distributed across, not from whatever column happens to be convenient.

shard key

hot shard

8.6 ADVANCED RELIABILITY Explain the CAP theorem, and why it's really a C-vs-A choice.

The CAP theorem states that a distributed data store can provide at most two of three guarantees simultaneously: **Consistency** (every read receives the most recent write, or an error), **Availability** (every request receives a non-error response, without guarantee it's the most recent write), and **Partition tolerance** (the system continues operating despite network messages being dropped or delayed between nodes). The reason this is really framed as a Consistency-vs-Availability choice, not a free pick of any two: in any real distributed system spanning more than one machine, network partitions *will* eventually happen — partition tolerance isn't a design choice you can opt out of, it's a physical reality you must design for. So the actual decision a distributed database makes is only relevant *during* an active partition: does it remain available and risk serving possibly-stale data (an AP choice), or does it refuse requests it can't guarantee are consistent, sacrificing availability until the partition heals (a CP choice)? "Choosing CA" isn't a real option for a genuinely distributed system — it only describes a single-node system, which by definition can't experience an internal network partition at all.

CAP theorem

partition tolerance isn't optional

8.7 ADVANCED RELIABILITY What is PACELC, and how does it extend CAP?

CAP only describes the trade-off *during a partition* — it says nothing about the trade-off a distributed system faces the rest of the time, under completely normal operation, which is a real and separate design decision. **PACELC** makes this explicit: **if Partitioned**, choose between **Availability** and **Consistency** (exactly CAP's choice) — **Else** (normal operation, no partition), choose between **Latency** and **Consistency**. Even with no partition at all, a system that wants strong consistency (e.g. synchronously confirming a write on every replica before acknowledging it, per 8.3) pays a real latency cost for that guarantee; a system that wants low latency has to accept some staleness risk on reads, partition or not. PACELC is the more complete, more practically useful framework precisely because most of a distributed database's actual operating time is spent in the "normal, no partition" state, where the L-vs-C trade-off, not the CAP trade-off, is the one actually governing day-to-day behavior.

PACELC latency-consistency trade-off

8.8 ADVANCED RELIABILITY

Scenario: the primary database goes down at 2am. Walk through the failover decision process.

1. **Confirm it's actually down**, not a transient network blip — a premature failover triggered by a false alarm is its own serious risk (see split-brain, below). Automated health checks typically require multiple consecutive failed checks before triggering.
2. **Identify the most caught-up replica**. If replication is asynchronous (8.3/8.4), different replicas may have slightly different amounts of applied WAL — promoting the most current one minimizes data loss.
3. **Promote that replica to primary**, and repoint application traffic to it (via DNS, a proxy, or a service-discovery update).
4. **Quantify what was actually lost** — per 8.4, the gap between the old primary's last committed write and what the promoted replica had actually received and applied is real, bounded, and should be measured and communicated, not assumed away.
5. **Guard against split-brain** — the single most dangerous failure mode here: if the "dead" original primary is not actually dead (just network-partitioned from the monitoring system) and comes back while still believing it's the primary, you can end up with *two* nodes both accepting writes independently, silently diverging — a fencing mechanism (forcibly ensuring the old primary can't accept writes again until it's confirmed demoted) is what prevents this, and its absence is the most common cause of a failover turning into a much worse incident than the original outage.

failover split-brain fencing

WHAT THE WEB SOURCES EMPHASIZE

PostgreSQL's own documentation on WAL and PITR, alongside multiple independent engineering-blog treatments of CAP/PACELC and sharding, converge on the same core warning: CAP is frequently mis-taught as "pick any two," when the accurate, docs-and-practitioner-backed framing is that partition tolerance is non-optional and the real choice is narrower than the popular simplification suggests.

Sources consulted

Compiled from current authoritative documentation, GATE archives, and aggregated practitioner/interview-report sources as of September 2026. Where this guide says a question is "commonly asked" or "frequently reported," that reflects language or data found directly in the cited source, not an invented popularity claim.

[PostgreSQL Documentation — Chapter 13: Concurrency Control \(MVCC, isolation levels\)](#)

[PostgreSQL Documentation — Continuous Archiving and Point-in-Time Recovery](#)

[PostgreSQL Documentation — Write Ahead Log](#)

[MySQL 8.0 Reference Manual — InnoDB Locking \(gap locks, next-key locks\)](#)

[GeeksforGeeks — Commonly Asked DBMS Interview Questions](#)

[GeeksforGeeks — Top 100 SQL Interview Questions](#)

[GeeksforGeeks — DBMS GATE CSE Previous Year Questions](#)

[Glassdoor — SQL Database Engineer Interview Questions](#)

[Towards Data Science — SQL Window Functions, Demonstrated with LeetCode Interview Questions](#)

[Hello Interview — Database Indexing for System Design Interviews](#)

[AlgoMaster — NULL Handling and COALESCE](#)

[Wikibooks — SQL NULLs and Three-Valued Logic](#)

[pgDash — Isolation Anomalies in PostgreSQL](#)

[DATAVERSITY — Applying CAP Theorem to Database Choice](#)